

cc writeup

August 14, 2026

1 Introduction

This document contains a brief explanation of how each task was programmed. JavaScript was chosen as a programming language since one of our goals from the beginning was to have all the tasks be available online and in an interactive format.

Several other languages were considered, notably Julia, Python and Rust however the setup for each language to run and be easily interactive on a website was deemed to outweigh any possible performance benefits. Thus, despite not being known as a scientific programming language we decided to learn and use JavaScript, since it was simple to add to a website.

Plotly was chosen as a plotting library over other alternatives, such as charts.js, since it had more built in interactivity and also looked more similar to other scientific plotting software, such as matplotlib or MATLAB.

2 Task 1: Random Walk

A class called `Particle` is first created, with a constructor that initialises a `Float64Array` and also chooses a random colour.

```
1 class Particle {
2   constructor(steps) {
3     this.path = new Float64Array((steps + 1) * 2);
4     this.length = 1;
5     this.colour = COLOURS[Math.floor(Math.random() * COLOURS.length)];
6   }
}
```

A flat `Float64Array` was chosen to help with performance, and path data is stored sequentially, ie `[x1, y1, x2, y2..]`, hence the length being double the steps. The `randomWalk()` method then first selects the correct index, using the `length` attribute, which effectively stores where in the path the particle currently is at. The code then uses the suggested method of randomly walking, and appends the new coordinates to the path and updates `length`

```
1   randomWalk() {
2     const path = this.path;
3     const idx = (this.length - 1) * 2;
4     const x = path[idx];
5     const y = path[idx + 1];
6     const direction = Math.random() * TWO_PI;
7     path[idx + 2] = x + Math.cos(direction);
8     path[idx + 3] = y + Math.sin(direction);
9     this.length++;
10  }
```

A set of helper methods are then defined to create an array of the x and y values

```
1   getXs() {
2     const xs = new Array(this.length);
3     for (let i = 0; i < this.length; i++) xs[i] = this.path[i * 2];
4     return xs;
5   }
6 }
```

```

7   getYs() {
8       const ys = new Array(this.length);
9       for (let i = 0; i < this.length; i++) ys[i] = this.path[i * 2 + 1];
10      return ys;
11  }

```

The function `generateParticles` then takes a number of particles, `n` and number of steps, `steps`, creates a new array, `particles`, of length `n`. It then iterates through the array, creates a new particle then iterates through every step and calls the `randomWalk()` method, generating a path for each particle before returning the final `particles` array.

```

1   function generateParticles(n, steps) {
2       const particles = new Array(n);
3       for (let i = 0; i < n; i++) {
4           const p = new Particle(steps);
5           for (let j = 0; j < steps; j++) {
6               p.randomWalk();
7           }
8           particles[i] = p;
9       }
10      return particles;
11  }

```

The following code consists of iterating through each particle, fetching their x and y values, and then plotting them using Plotly.

Whenever the function `restart()` is called, by pressing the button on the webpage, the plots are built again, with values `n` and `steps` being fetched from the webpage

```

1   function restart() {
2       particles = generateParticles(
3           document.getElementById('particleCount').value,
4           document.getElementById('stepCountInput').value
5       );
6       data = generateData(particles);
7
8       Plotly.newPlot('randomWalk', data, layout);
9   }

```

3 Brownian Motion

Since the code uses custom position and velocity objects, and implementing operator overloading was beyond the scope of my JavaScript knowledge several helper functions were defined for dealing with arithmetic that included the custom 'vector' objects.

```

1   function distance(p1, p2) {
2       return Math.sqrt((p1.x - p2.x) ** 2 + (p1.y - p2.y) ** 2);
3   }
4
5   function dot(a, b) {
6       return a.x * b.x + a.y * b.y;
7   }
8
9   function vecScale(v, s) {
10      return { x: v.x * s, y: v.y * s };
11  }
12
13  function vecAdd(a, b) {
14      return { x: a.x + b.x, y: a.y + b.y };
15  }
16
17  function vecSub(a, b) {
18      return { x: a.x - b.x, y: a.y - b.y };
19  }

```

The collision logic was handled using the zero-momentum frame, using a function that would take two particle objects, apply the equations and then update each of the particles velocities.

```
1  function collision(p1, p2) {
2      let d = distance(p1.position, p2.position);
3      let normal = {
4          x: (p1.position.x - p2.position.x) / d,
5          y: (p1.position.y - p2.position.y) / d
6      };
7      let totalMass = p1.mass + p2.mass;
8
9      let v_com = {
10         x: (p1.mass * p1.velocity.x + p2.mass * p2.velocity.x) / totalMass,
11         y: (p1.mass * p1.velocity.y + p2.mass * p2.velocity.y) / totalMass
12     };
13
14     let v1_frame = vecSub(p1.velocity, v_com);
15     let v2_frame = vecSub(p2.velocity, v_com);
16
17     let v1_frame_n = dot(v1_frame, normal);
18     let v2_frame_n = dot(v2_frame, normal);
19
20     let v1_frame_n_after = -v1_frame_n;
21     let v2_frame_n_after = -v2_frame_n;
22
23     let v1_frame_after = vecAdd(vecSub(v1_frame, vecScale(normal, v1_frame_n)),
24     ↪   vecScale(normal, v1_frame_n_after));
25     let v2_frame_after = vecAdd(vecSub(v2_frame, vecScale(normal, v2_frame_n)),
26     ↪   vecScale(normal, v2_frame_n_after));
27
28     p1.velocity = vecAdd(v1_frame_after, v_com);
29     p2.velocity = vecAdd(v2_frame_after, v_com);
30 }
```

The same `Particle` object from the previous task was used with some additional methods and attributes. Mass, radius, position, velocity and a collision flag were added, alongside some utility methods related to the array.

```
1  class Particle {
2      constructor(x, y) {
3          this.position = { x: x, y: y };
4          this.velocity = { x: 0, y: 0 };
5
6          this.radius = 0.3;
7          this.mass = 1;
8          this.hasCollided = false;
9
10         this.path = new Float64Array((t_steps + 1) * 2);
11         this.length = 1;
12         this.path[0] = x;
13         this.path[1] = y;
14     }
15
16     pushPoints(x, y) {
17         const idx = this.length * 2;
18         this.path[idx] = x;
19         this.path[idx + 1] = y;
20         this.length++;
21     }
22
23     getPoint(idx) {
24         return { x: this.path[idx * 2], y: this.path[idx * 2 + 1] };
25     }
26 }
```

25

}

Importantly, two new methods were added for moving the particle, `nextStep()` and `collisionStep()`. `nextStep()` uses a simple Euler integrator to update the particles position, and is used when 'physical' motion is desired.

`collisionStep()` is a variant of `randomWalk`. A helper function called `generateStep()` which takes the code logic for generating a new step has been defined and moved, so that `randomWalk` and `collisionStep()` can both call it.

`collisionStep()` continually generates new coordinates until they don't collide with the `otherParticle`. This prevents the collision detection code from firing repeatedly if the random walk chose coordinates within the other particle.

```

1  nextStep() {
2      this.position = { x: this.position.x + this.velocity.x, y: this.position.y +
3      ↪ this.velocity.y };
4      this.pushPoints(this.position.x, this.position.y);
5  }
6
7  collisionStep(otherParticle) {
8      let { newx, newy } = generate_step(this.position.x, this.position.y);
9      while (distance({ x: newx, y: newy }, otherParticle.position) < this.radius +
10      ↪ otherParticle.radius) {
11          ({ newx, newy } = generate_step(this.position.x, this.position.y));
12      }
13
14      this.pushPoints(newx, newy);
15      this.velocity = { x: newx - this.position.x, y: newy - this.position.y };
16      this.position = { x: newx, y: newy };
17  }

```

The class `bigParticle` inherits from `Particle` and overrides some values in the constructor to give it a larger mass and radius.

```

1  class bigParticle extends Particle {
2      constructor(x, y) {
3          super(x, y);
4          this.radius = 1;
5          this.mass = 2;
6      }

```

Under the main simulation function, `runSimulation()` the particles are first created. A `bigParticle` is created, spawning at the origin, whilst an array `particles` of length `number_of_particles`, which is fetched from the webpage, is initialised.

Each particle is iterated over, and a set of coordinates is generated, placing the particle randomly within a box, with dimensions decided by `VIEW_SIZE`.

```

1  let bigP = new bigParticle(0, 0);
2  let particles = new Array(number_of_particles);
3
4  for (let i = 0; i < number_of_particles; i++) {
5      let x = Math.random() * VIEW_SIZE - VIEW_SIZE / 2;
6      let y = Math.random() * VIEW_SIZE - VIEW_SIZE / 2;
7
8      while (distance({ x: x, y: y }, bigP.position) < bigP.radius + 0.5) {
9          x = Math.random() * VIEW_SIZE - VIEW_SIZE / 2;
10         y = Math.random() * VIEW_SIZE - VIEW_SIZE / 2;
11     }
12
13     let particle = new Particle(x, y);
14     particles[i] = particle;
15 }

```

Under the main simulation loop, each particle is iterated over, checked if it is colliding with the big particle and if it is the collision is handled, the particle calls the method `collisionStep()` and its flag `hasCollided` is updated.

```

1   for (let i = 0; i < t_steps; i++) {
2       for (const particle of particles) {
3           if (isColliding(particle, bigP)) {
4               console.log("collision detected at step", i, "with particle",
5                           ↳ particle);
6               collision(particle, bigP);
7               particle.collisionStep(bigP);
8               particle.hasCollided = true;
9           }
10      }

```

Each particle is then iterated over again, and if its `hasCollided` flag is false it calls `randomWalk()`, this is so that any particle that has collided does not move two steps in one simulation step. The big particle then moves physically using `nextStep()`

```

1   for (const particle of particles) {
2       if (!particle.hasCollided) {
3           particle.randomWalk();
4       } else {
5           particle.hasCollided = false;
6       }
7   }
8
9   bigP.nextStep();
10  }

```

The remaining code is related to fetching the x and y data from each particle and plotting it, using `requestAnimationFrame()` to animate it.

4 Photoelectric simulation

The simulation recycles the functions used for the physics in plotting the graphs. The code also implements a helper function, called `wavelengthToRGB()` whose values were based of a function found online.

Important information about the simulation is stored in a `state` object:

```

1   state = {
2       voltage: 0,
3       wavelength: 400,
4       intensity: 50,
5       colour: {r: 255, g: 0, b: 0},
6       workFunction: 4.3
7   }

```

A series of `.addEventListener` methods on each of the inputs then updates the `state` and calls a function `updateValues()`.

```

1   document.getElementById("voltage").addEventListener("input", function (event) {
2       state.voltage = +event.target.value
3       updateValues()
4   })
5   document.getElementById("wavelength").addEventListener("input", function (event) {
6       state.wavelength = +event.target.value
7       updateValues()
8   })
9   document.getElementById("intensity").addEventListener("input", function (event) {
10      state.intensity = +event.target.value
11      updateValues()
12  })
13  document.getElementById("material").addEventListener("change", function (event) {
14      state.workFunction = +event.target.value
15      updateValues()
16  })

```

`updateValues()` does as it says, uses the previous equations and updates the values on the webpage.

The next section of code is responsible for drawing the canvas. The html `canvas` element is used, and a series of functions are defined that draw the lamp, beam anode and cathode and all the electrons. A final function `drawStage()` wraps all these functions and is called within the main simulation loop.

The main aspect of the simulation is the electrons and their motion. A function, `spawnElectrons()` is first defined, checks if its energy is valid to spawn an electron and then creates an `electron` object. `electron` Has an initial x-coordinate on the cathode with a small offset and a random y-coordinate. It has a random fraction `u` of the maximum energy and is initialised with an undefined velocity.

```

1  function spawnElectron() {
2      ke = photonEnergyEV(state.wavelength) - state.workFunction;
3      if (ke <= 0) { return }
4      const u = Math.random();
5      electrons.push({
6          x: cathodeX + 4,
7          y: Math.random() * (plateBottom - plateTop - 6) + plateTop + 3,
8          u: u,
9          v: undefined,
10         t: 0,
11         turned: false
12     });
13
14 }

```

The function `stepElectrons()` contains the main logic for the simulation. We first get the current kinetic energy and calculate two constants called `speedScale` and `accelScale` which are used to scale the simulation for animation purposes. Since the velocity of an electron is

$$v = \sqrt{\frac{2e}{m_e}} \cdot \sqrt{E_{eV}} \quad (1)$$

We can say

$$v \propto \sqrt{E_{eV}}$$

and replace the constant $\sqrt{\frac{2e}{m_e}}$ with `speedScale` so that the simulation can be animated properly. In order to remain physical the acceleration scale is derived. Consider the previous equation, where $S = \text{speedScale}$, but squared so that it is an equation of energy instead:

$$v^2 = S^2 E_{eV}$$

By considering the acceleration the electron experiences through distance d , between the cathode and anode we can say that the work done, or E_{eV} , is simply V . We can therefore say that

$$\Delta v^2 = S^2 \cdot V$$

Using the kinematic equation

$$v^2 - u^2 = 2ad$$

We can arrive at

$$2ad = S^2 \cdot V$$

and rearranging for a we arrive at

$$a = \frac{S^2}{2d} \cdot V \quad (2)$$

and so our `accelScale` = $\frac{S^2}{2d}$ The function then iterates through every `electron` object stored in the global array `electrons` calculates and updates their positions, again using a simple Euler integrator. If the electron has 'turned' a flag is updated, so that when being rendered it is drawn as red. If the electron has reached either the cathode or anode or has been alive too long it is deleted.

```

1  function stepElectrons(dt) {
2      const ke = photonEnergyEV(state.wavelength) - state.workFunction;
3      const voltage = state.voltage;
4      const d = anodeX - cathodeX;
5

```

```

6   const speedScale = 100;
7   const accelScale = (speedScale ** 2) / (2 * d);
8   const a = accelScale * voltage;
9
10  const next = []
11  for (const el of electrons) {
12      const eEnergy = ke * el.u;
13      if (el.v === undefined) {
14          el.v = speedScale * Math.sqrt(eEnergy);
15      }
16
17      el.v += a * dt;
18      el.x += el.v * dt;
19      el.t += dt;
20
21      if (el.v <= 0) {
22          el.turned = true
23      } else {
24          el.turned = false
25      }
26
27      if (el.x >= anodeX - 6) {
28          currentEvents.push(simtime)
29          continue;
30      }
31      if (el.x <= cathodeX + 2 && el.v <= 0) continue;
32      if (el.t > 600) continue;
33      next.push(el);
34  }
35  electrons = next;
36 }

```

A function, `updateCurrent` counts the number of electrons over a certain sample window and then calculates a physical representation of the current, scaled since the actual value is very low.

```

1   function updateCurrent(dt) {
2       simtime += dt
3       while (currentEvents.length > 0 && currentEvents[0] < simtime - sampleWindow)
4           → {
5               currentEvents.shift()
6           }
7       document.getElementById("current").textContent = ((currentEvents.length /
8           → sampleWindow) * e * 1e18).toFixed(2)
9   }

```

Finally, using `requestAnimationFrame`, a `tick()` function is used to animate. Both the rate and count of electron spawning is determined by the intensity.

```

1   function tick(timestamp) {
2       frame++;
3       if (lastTime === null) lastTime = timestamp;
4       const dt = Math.min((timestamp - lastTime) / 1000, 0.1);
5       lastTime = timestamp;
6
7       const ke = photonEnergyEV(state.wavelength) - state.workFunction;
8       if (ke > 0) {
9           const rate = Math.max(1, Math.round(25 - state.intensity * 0.2));
10          const count = 1 + Math.floor(state.intensity / 45);
11          if (frame % rate === 0) {
12              for (let i = 0; i < count; i++) spawnElectron();
13          }
14      }

```

```

15     stepElectrons(dt);
16     updateCurrent(dt);
17     drawStage();
18     requestAnimationFrame(tick);
19 }

```

5 Task 3, 5, 6, 7 and 9: Blackbody spectra; Emission spectra; Electron diffraction; Particle in a box and Compton scattering

The code for each of these tasks is very similar in structure so I have grouped them under one section. Code from Task 3: Blackbody spectra will be used as a demonstration.

Firstly a set of x values would be defined with a given range and step, since JavaScript lacks a built in `linspace` function, a simple for loop was used instead:

```

1     let wavelengths = [];
2     for (let wl = 10; wl <= 2500; wl += 1) {
3         wavelengths.push(wl);
4     }

```

Then a function containing the relevant equation would be defined, in this case Planck's equation.

```

1     function planck(wavelength, temperature) {
2         const lambda = wavelength * 1e-9;
3         return ((2* h* c**2)/(lambda ** 5))*(1/(Math.exp((h* c)/(lambda* k_b*
4             temperature))-1))/1e-4;
5     }

```

Then within the `generateData()` function the array of x values would be iterated over, the aforementioned function would be called on the value to generate an appropriate y value, of which both would be appended to a respective x-value and y-value array and then constructed into an object to be handed off to Plotly.

```

1     const x = [];
2     const y = [];
3     for (const wl of wavelengths) {
4         x.push(wl);
5         y.push(planck(wl, temp));
6     }
7     data.push({
8         x: x,
9         y: y,
10        mode: 'lines',
11        name: `T = ${temp} K`,
12        line: { color: `hsl(${(temp - 3000) / 5000 * 240}, 100%, 50%)` }
13    });

```

If the equation depended on two values, in this case `temperature`, another array of values could be defined

```

1     temperatures = [3000, 4000, 5000, 6000, 7000, 8000];

```

and iterated over within the `generateData()` function.

For each of these tasks this structure was applied to generate the plots.

6 Task 8: Quantum Cryptography Calculator

The code for this task is very similar to the structure for the aforementioned tasks, except that it includes an interactive, movable marker.

A function `update()` is defined which is called each time a change in the sliders is detected. This function parses the values of theta and phi, calculates the quantum and classical values, as well as the difference, updates the values of the labels on the website and then calls `Plotly.restyle` to redraw the graph with the marker updated at the new position


```

1  function update() {
2      const theta = parseFloat(thetaSlider.value);
3      const phi = parseFloat(phiSlider.value);
4      thetaVal.textContent = theta;
5      phiVal.textContent = phi;
6
7      const classical = classicalMismatch(theta, phi);
8      const quantum = quantumMismatch(theta, phi);
9      const difference = quantum - classical;
10
11     document.getElementById('classicalVal').textContent = classical.toFixed(4);
12     document.getElementById('quantumVal').textContent = quantum.toFixed(4);
13     document.getElementById('differenceVal').textContent = difference.toFixed(4);
14
15     Plotly.restyle('difference', { x: [[theta]], y: [[phi]], z: [[difference]] },
16         ↪ [1]);
17     Plotly.restyle('classical', { x: [[theta]], y: [[phi]], z: [[classical]] },
18         ↪ [1]);
19     Plotly.restyle('quantum', { x: [[theta]], y: [[phi]], z: [[quantum]] }, [1]);
20 }

```

7 2D Orbitals

We used the given solutions to the Schrödinger Equation. In order to display a heatmap of values a grid is used. The function `initialiseGrid()` is used to generate a grid, with a size based of the value of `n` and a quality, ie number of pixels, determined by the `quality` slider on the webpage. The grid then calculates the polar coordinates for each and stores it all in the `grid` object.

```

1  let GRID_SIZE = document.getElementById('quality').value * 100;
2  let EXTENT;
3  const grid = {
4      x: null,
5      y: null,
6      r: null,
7      azi: null,
8      elev: null
9  };
10
11  function initialiseGrid(plane = 'xy', displacement = 0) {
12
13      grid.x = new Float64Array(GRID_SIZE);
14      grid.y = new Float64Array(GRID_SIZE);
15      grid.z = new Float64Array(GRID_SIZE);
16      grid.r = new Array(GRID_SIZE);
17      grid.azi = new Array(GRID_SIZE);
18      grid.elev = new Array(GRID_SIZE);
19
20      for (let i = 0; i < GRID_SIZE; i++) {
21          grid.x[i] = -EXTENT + 2 * EXTENT * i / (GRID_SIZE - 1);
22          grid.y[i] = -EXTENT + 2 * EXTENT * i / (GRID_SIZE - 1);
23          grid.z[i] = -EXTENT + 2 * EXTENT * i / (GRID_SIZE - 1);
24      }
25
26      for (let j = 0; j < GRID_SIZE; j++) {
27
28          grid.r[j] = new Float64Array(GRID_SIZE);
29          grid.azi[j] = new Float64Array(GRID_SIZE);
30          grid.elev[j] = new Float64Array(GRID_SIZE);
31          for (let i = 0; i < GRID_SIZE; i++) {
32
33              let x = grid.x[i]

```

```

34     let y = grid.y[j]
35     let z = grid.z[j]
36
37     if (plane === 'xy') {
38         x = grid.x[i]
39         y = grid.y[j]
40         z = displacement
41     } else if (plane === 'xz') {
42         x = grid.x[i]
43         y = displacement
44         z = grid.y[j]
45     } else if (plane === 'yz') {
46         x = displacement
47         y = grid.x[i]
48         z = grid.y[j]
49     }
50     const r = Math.sqrt(x * x + y * y + z * z);
51     const azi = Math.atan2(y, x);
52     const elev = r === 0 ? 0 : Math.asin(z / r);
53
54     grid.r[j][i] = r;
55     grid.azi[j][i] = azi;
56     grid.elev[j][i] = elev;
57 }
58 }
59 }

```

A function then simply iterates through each point of the grid, calculates the wavefunction at that point, and stores the value in a separate array. Two versions of the wavefunction are plotted, one being the real component, and the other being the probability. When plotting each value is divided by the maximum value to "normalise" the plot.

```

1 function computeOrbitalAndDensityMaps(
2     Z,
3     A,
4     n,
5     L,
6     M,
7     plane = 'xy'
8 ) {
9     const orbital = [];
10    const density = [];
11    let maxAbs = 0;
12    let maxVal = 0;
13
14    for (let j = 0; j < GRID_SIZE; j++) {
15        orbital[j] = new Float64Array(GRID_SIZE);
16        density[j] = new Float64Array(GRID_SIZE);
17
18        for (let i = 0; i < GRID_SIZE; i++) {
19            const wf = compute_wavefunction(
20                grid.r[j][i],
21                grid.azi[j][i],
22                grid.elev[j][i],
23                Z,
24                A,
25                n,
26                L,
27                M,
28                plane
29            );
30

```

```

31     const realPart = wf.psi.re;
32     const probability = wf.probability;
33
34     orbital[j][i] = realPart;
35     density[j][i] = probability;
36
37     const absValue = Math.abs(realPart);
38     if (absValue > maxAbs) {
39         maxAbs = absValue;
40     }
41     if (probability > maxVal) {
42         maxVal = probability;
43     }
44 }
45 }
46
47 for (let j = 0; j < GRID_SIZE; j++) {
48     for (let i = 0; i < GRID_SIZE; i++) {
49         orbital[j][i] /= maxAbs;
50         density[j][i] /= maxVal;
51     }
52 }
53
54 return { orbital, density };
55 }

```

For interactivity the previously described methods are used, where an `addEventListener` method listens for any change to the input values, and calls `updateValues()` which updates the plots and labels.

8 3D Orbitals

The same equations for the wavefunction are used, but instead of using a grid method we instead chose to use a rejection-sampling based Monte Carlo model instead. An initial utility function performs a coarse pass over the selected area and estimates the max probability.

```

1 function estimateMaxProb(Z, A, n, L, M, resolution) {
2     const grid = new Float64Array(resolution);
3     for (let i = 0; i < resolution; i++) {
4         grid[i] = -EXTENT + 2 * EXTENT * i / (resolution - 1);
5     }
6
7     let maxProb = 0;
8     for (let i = 0; i < resolution; i++) {
9         let x = grid[i];
10        for (let j = 0; j < resolution; j++) {
11            let y = grid[j];
12            for (let k = 0; k < resolution; k++) {
13                let z = grid[k];
14                const r = Math.sqrt(x * x + y * y + z * z);
15                const azi = Math.atan2(y, x);
16                const elev = r === 0 ? 0 : Math.asin(z / r);
17
18                const wf = compute_wavefunction(r, azi, elev, Z, A, n, L, M);
19                const prob = wf.probability;
20                if (prob > maxProb) {
21                    maxProb = prob;
22                }
23            }
24        }
25    }
26    return maxProb;

```

}

This then is used in the next function, where a set of random x, y and z coordinates within the extent are chosen. The normalised probability at that point is then compared to a random number, and if the random number is less than the probability at that point it is plotted. This means that areas with lower probability will have less points plotted and areas with higher probability will have more points plotted. This is then repeated until either upper bound on trials or target number of points is reached. Each point is subsequently properly normalised with the real max probability once all points have been computed.

```

1 function compute3DPointCloud(Z, A, n, L, M, targetPoints, maxTrials) {
2   let maxProb = estimateMaxProb(Z, A, n, L, M, 20);
3   const px = [];
4   const py = [];
5   const pz = [];
6   const pAmp = [];
7   let trials = 0;
8   while (px.length < targetPoints && trials < maxTrials) {
9     trials++;
10    const x = Math.random() * 2 * EXTENT - EXTENT;
11    const y = Math.random() * 2 * EXTENT - EXTENT;
12    const z = Math.random() * 2 * EXTENT - EXTENT;
13
14    const r = Math.sqrt(x * x + y * y + z * z);
15    const azi = Math.atan2(y, x);
16    const elev = r === 0 ? 0 : Math.asin(z / r);
17    const wf = compute_wavefunction(r, azi, elev, Z, A, n, L, M);
18    if (Math.random() < wf.probability / maxProb && wf.probability > 0) {
19      px.push(x);
20      py.push(y);
21      pz.push(z);
22      pAmp.push(wf.psi.re);
23    }
24  }
25  for (let i = 0; i < px.length; i++) {
26    if (Math.abs(pAmp[i]) > maxProb) {
27      maxProb = Math.abs(pAmp[i]);
28    }
29  }
30
31  for (let i = 0; i < px.length; i++) {
32    pAmp[i] /= maxProb;
33  }
34  return { x: px, y: py, z: pz, amp: pAmp, trials };
35 }

```

The points are then plotted, with their colour also encoding the probability of the respective point, on a colour map scale. The same code as before is used to handle any updates to the values.